

CI/CD

Integración y Despliegue Continuo — GitHub Actions, Vercel y flujo completo profesional

CI — Continuous Integration (Integración Continua): automatiza la verificación del código.

CD — Continuous Delivery/Deployment (Entrega/Despliegue Continuo): automatiza el despliegue.

El problema que resuelven

Sin CI/CD el proceso es manual y propenso a errores:

```
Sin CI/CD:  
Escribes código → subes al repo → alguien revisa manualmente  
→ ejecuta tests manualmente → construye manualmente  
→ despliega manualmente → rezas para que no haya roto nada  
  
Con CI/CD:  
Escribes código → push → todo ocurre automáticamente
```

CI — Integración Continua

Cada vez que subes código al repositorio se ejecutan automáticamente:

- **Tests** — unitarios, de integración, end-to-end
- **Linting** — comprobación de estilo de código
- **Type checking** — TypeScript sin errores
- **Build** — que la app compila correctamente

Si algo falla, el merge se bloquea. Nadie puede meter código roto en la rama principal.

```
Developer hace push  
↓  
CI detecta el push  
↓  
Ejecuta tests → ¿pasan? ■ → puede hacer merge  
                ■ → merge bloqueado, avisa al developer
```

CD — Despliegue Continuo

Una vez que el código pasa el CI y se mergea, se despliega automáticamente.

- **Continuous Delivery** — el despliegue está listo pero requiere aprobación manual
- **Continuous Deployment** — el despliegue ocurre completamente solo

```
Merge a main  
↓  
CI pasa ■  
↓  
Build automático  
↓  
Deploy automático a producción
```



Tu app está actualizada

GitHub Actions — el más popular

Se configura con archivos YAML en `.github/workflows/`. Integrado directamente en GitHub, gratuito para repositorios públicos.

CI básico para Next.js

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npx tsc --noEmit
      - run: npm run lint
      - run: npm run test
      - run: npm run build
    env:
      DATABASE_URL: ${ secrets.DATABASE_URL }
```

CI + CD completo

```
# .github/workflows/deploy.yml
name: CI/CD

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npm run lint
      - run: npm run test
      - run: npm run build

  deploy:
    needs: test # espera a que test pase
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy a Vercel
```

```
uses: amondnet/vercel-action@v25
with:
  vercel-token: ${ secrets.VERCEL_TOKEN }
  vercel-org-id: ${ secrets.VERCEL_ORG_ID }
  vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }
  vercel-args: '--prod'
```

Entornos — el flujo completo profesional

Un proyecto profesional suele tener varios entornos, cada uno con su propio deploy automático:

develop branch	→	entorno de desarrollo	→	dev.tuapp.com
staging branch	→	entorno de staging	→	staging.tuapp.com
main branch	→	producción	→	tuapp.com

Checks típicos en un CI

```
steps:
  - run: npm run lint           # ESLint – estilo de código
  - run: npx tsc --noEmit       # TypeScript – tipos correctos
  - run: npm run test           # Jest/Vitest – tests unitarios
  - run: npm run test:e2e       # Playwright – tests end-to-end
  - run: npm run build          # que compila sin errores
  - run: npm audit              # vulnerabilidades de seguridad
```

Secrets — variables de entorno en CI

Las variables sensibles no van en el código — se configuran en GitHub y se inyectan en el workflow. Se configuran en: *Repositorio* → *Settings* → *Secrets and variables* → *Actions*

```
# En el workflow
env:
  DATABASE_URL: ${ secrets.DATABASE_URL }
  NEXTAUTH_SECRET: ${ secrets.NEXTAUTH_SECRET }
```

Vercel como CD automático

Si usas Vercel, ya tienes CD sin configurar nada. Solo necesitas añadir el CI con GitHub Actions.

Push a main	→	deploy automático a producción
Push a rama X	→	deploy automático a preview URL
Pull Request	→	deploy de preview + comentario en el PR

Herramientas de CI/CD

Herramienta	Descripción	Ideal para
GitHub Actions	Integrado en GitHub, muy popular	Cualquier proyecto en GitHub
GitLab CI	Integrado en GitLab	Proyectos en GitLab
CircleCI	Muy rápido, buena caché	Empresas medianas
Jenkins	Self-hosted, muy configurable	Empresas grandes con servidor propio
Vercel	CD automático sin configurar	Next.js — hace el deploy solo
Netlify	Similar a Vercel	Proyectos frontend

En resumen

CI = automatizar la verificación del código

CD = automatizar el despliegue del código

CI/CD = push → tests → build → deploy

todo automático, sin intervención humana